```
redirect_to action: "show", id: product.id
```

One more data type is worth mentioning: the regular expression.

Regular Expressions

A regular expression lets you specify a *pattern* of characters to be matched in a string. In Ruby, you typically create a regular expression by writing `/pattern/` or `%r{pattern}`.

For example, we can use the regular expression `/Perl|Python/` to write a pattern that matches a string containing the text *Perl* or the text *Python*.

The forward slashes delimit the pattern, which consists of the two things that we're matching, separated by a vertical bar (`|`). The bar character means either the thing on the left or the thing on the right—in this case, either *Perl* or *Python*. You can use parentheses within patterns, just as you can in arithmetic expressions, so we could also write this pattern as `/P(erl|ython)/`. Programs typically use the `=~` match operator to test strings against regular expressions:

```
if line =~ /P(erl|ython)/
  puts "There seems to be another scripting language here"
end
```

You can specify *repetition* within patterns. `/ab+c/` matches a string containing an *a* followed by one or more *b*s, followed by a *c*. Change the plus to an asterisk, and `/ab*c/` creates a regular expression that matches one *a*, zero or more *b*s, and one *c*.

Backward slashes start special sequences; most notably, `\d` matches any digit, `\s` matches any whitespace character, and `\w` matches any alphanumeric (*word*) character, `\A` matches the start of the string and `\z` matches the end of the string. A backslash before a wildcard character, for example `\.`, causes the character to be matched as is.